GOVERNANCE SYSTEM / FINAL SPECIFICATION

法案系统
建设说明书

轻量治理闭环版

版本 FINAL-V3.2 | 轻量治理闭环版

冻结 FROZEN-FINAL-V3.2

修订 2026-08-24

Markdown 唯一权威源 · PDF 非权威派生版

法案系统建设说明书

文件版本：FINAL-V3.2 | 轻量治理闭环版 冻结目标版本：FROZEN-FINAL-V3.2 修订日期：2026-08-23
文件性质：治理系统源头建设规范 / Governance Bootstrap Specification 文件定位：规定法案正文、lawId、唯一 Owner、唯一 registry、任务范围、证据等级、渐进迁移和冻结发布 适用对象：Codex、AI Coding Agent、开发者、架构师、审计者、后续自动化执行者 交付形态：Markdown 为唯一权威源；PDF 为可选非权威派生物，仅在 GOVERNANCE-FREEZE.json 明确启用时生成并绑定 冻结状态：SPEC_FROZEN 状态只由当前有效 GOVERNANCE-FREEZE.json 对本文件字节的 SHA-256 绑定判定；本文件正文不自我声明当前冻结状态
冻结声明：本文件属于冻结产品，默认只读；只有人类明确授权修改本文件时才允许修改。 法条标识：全部 lawId 固定使用中性前缀 LAW-；任何历史项目名称、旧 fence 名称或旧 lawId 前缀不得进入本文件、法案正文、generated registry、证据或报告。

0. 本说明书的唯一身份

本说明书不是业务法案，不定义某个产品应该具备什么功能，也不替代项目的基础法案、功能法案、施工图、规格、契约或代码。

本说明书只拥有一个主权：

TXT

```
定义“法案系统本身必须如何建设、如何运行、如何防止 Codex 漂移、如何阻断第二链路”。
```

本说明书必须让一个从未接触项目的 Codex 在读完后能够直接回答并执行：

TXT

```
法案放在哪里；
法条按什么结构编写；
什么内容属于基础法案；
什么内容属于功能法案；
lawId 怎么编号和退役；
Owner 怎么唯一绑定；
唯一 registry 怎么生成和验真；
READ_ONLY、CHANGE、READY、RELEASE 如何分级；
CHANGE 在落笔前记录什么边界；
哪些范围扩张和迁移机制合法；
哪些越界、双重主权和伪 PASS 必须禁止；
质量系统怎么产出一份逐法条报告；
Exam 和 Dynamic Proof 在什么情况下必须存在；
什么证据足以宣布 PASS；
什么情况必须停止并返回 FAIL；
最终如何冻结，避免本文件被 Codex 自行修改。
```

本说明书不是可供
Codex“参考发挥”的原则集合。所有出现“固定”“必须”“只能”“不得”“精确”的条款均约束其明确点名的外部合同；Codex 不得改写这些法律效果。第 5 节、第 6 节第 9 项和第 68 节明确交给仓库的内部实现不因此被冻结。

第一部分 | 最高秩序与不可变公式

1. 法案系统的七件套

完整法案系统固定由以下七类对象组成：

TXT

1. 冻结的《法案系统建设说明书》；
2. 一份《基础法案.md》；
3. 一份《功能法案.md》；
4. 由唯一 compiler 生成的 law registry；
5. 每个 capabilityDomain 唯一的 Owner Manifest；
6. 一套普通治理测试；
7. 对当前候选逐 lawId 产生裁决的一份质量报告。

唯一公式固定为：

TXT

```
法案系统
=
权威正文
+ 明确边界
+ 唯一 Owner
+ 完整索引
+ 任务范围控制
+ 可复放证据
+ 失败关闭执法
```

以下组合均不构成法案系统：

TXT

```
只有文档，没有机器执法；
只有测试，没有 active lawId；
只有索引，没有权威正文；
只有 Owner 名称，没有路径主权；
只有正向目标，没有禁止面；
只有 PASS 汇总，没有逐法条报告；
只有 Codex 提示词，没有 active 法案；
只有目录和空壳，没有真实检查；
只有静态检查，却宣称真实运行已经成立。
```

2. 每条规则必须同时回答五个问题

每条 active lawId 必须精确回答：

TXT

```
要什么；
不要什么；
谁负责；
怎么证明；
违反后怎么办。
```

任何不能写出明确“不要什么”的要求，不得进入 active 法案。它只能继续讨论，不能取得治理主权。

2.1 边界闭合的唯一判定

“说清楚”固定等于同时写清正边界和负边界，不能只写目标，也不能只写禁令：

治理对象	要：必须做	不要：绝对禁区
法案	summary、唯一 Owner、required evidence、违法后果	forbidden[]、不得被下级覆盖
施工图	按需记录复杂拓扑、合同、状态和恢复	空壳、重复法案、第二 Owner/第二真相
规格	按需记录代码之外的字段、状态、错误和 contract identity	空模板、重复法案、隐式状态

治理对象	要：必须做	不要：绝对禁区
registry	忠实投影 active law、Owner、source 和按需合同	新增规范、人工维护、PASS/READY、第二索引
质检	正确证据类型、完整分母、一份逐法条报告、失败关闭	证据互相冒充、漏项放行、warning/partial/fake PASS
任务控制	READ_ONLY 零写入、CHANGE 轻记录、RELEASE 候选绑定	未声明写入、候选漂移、等级冒充和长期双重真相

边界判定规则固定：

1. “没有写不要”不等于默认禁止；它表示边界未闭合，必须 FAIL。
2. “没有写要”不等于执行者可以自行补全；它表示生产义务未定义，必须 FAIL。
3. 合法可选项必须进入精确枚举并同时写明选择者、前置条件和失败语义；不得把“可选”当作自由发挥区。
4. 同一对象在两个位置出现不同的必做、禁区、状态、字段、错误码或输出位置时，不得择一实现；必须先消除规范冲突。

3. 终态与建设阶段

active 法案描述需要长期成立的生产规则；施工可以分步进行。

阶段、MVP、preview、阶段性 feature flag 和迁移窗口可以合法存在，但必须标明：

```TXT
当前完成范围；
未完成范围；
交付等级；
适用人群或流量；
进入下一等级的条件；
临时机制存在时的删除日期。
```

不得把阶段成果冒充完整生产终态，不得把未实现或未证明的能力记为 PASS。

4. 权威顺序与两类人类授权

必须把“人类授权”拆成两种不同主权，禁止混为一谈。

4.1 本次任务范围主权

人类对本次任务的原文只决定：

```TXT
本次目标；
允许触碰的对象；
是否允许 law-amendment；
是否允许 source-spec-amendment；
是否为空白建制或既有治理迁移；
本次等级是 READ_ONLY、CHANGE、READY 还是 RELEASE；
是否涉及跨仓版本化合同。
```

任务原文不得直接降低现役法案、改变 Owner、跳过证据或授权第二链路。若普通任务原文与 active law 冲突，固定结果为 USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW。

4.2 规范内容主权

项目规范顺序固定为：

TXT

```
冻结的《法案系统建设说明书》
-> 《基础法案.md》
-> 《功能法案.md》
-> 按需系统或 capabilityDomain 施工图
-> 按需规格与机器契约
-> 本次 CHANGE 范围记录或 RELEASE 候选绑定
-> Owner Manifest 与唯一 law registry
-> 实现代码
-> 当前候选证据
-> 历史记录与讨论材料
```

解释规则固定为：

1. 人类明确授权“修订法案”只打开被点名法条及法定闭包的写权限；新内容仍必须符合本说明书。
2. 下级对象与上级冲突时，修改或删除下级对象；迁移期兼容机制只有满足第 43.2 节才合法。
3. 当前代码永远不能反向证明法案应该降低。
4. generated artifact 只投影，报告只证明，消费端副本只缓存；三者均不得取得源头主权。
5. 历史报告、旧提示词、工单、会议纪要和 Git 历史只用于追溯，不能取得当前主权。

第二部分 | 固定技术底座与目录

5. 实现技术由仓库决定

除第 68 节固定的五个公共治理命令身份外，本说明书不指定编程语言、运行时、依赖库、包管理器、测试框架或内部命令。每个仓库沿用与自身业务相容的技术栈，并只保留一个现役治理实现和一个 canonical quality 入口。

工具版本、依赖版本和锁文件由仓库自己的工程配置管理。更换实现技术不得建立第二治理主链，也不得改变本说明书规定的权威、Owner、范围、证据和交付边界。

projectChecks 由仓库按实际构建、类型、测试和部署需要定义，不取得 law verdict。CHANGE 只运行与本次差异相关的检查；READY 和 RELEASE 再运行各自要求的完整检查。

6. 最小固定路径

基础路径：

TXT

```
AGENTS.md

.github/instructions/agent-rules/
  基础法案.md
  功能法案.md

docs/governance/
  法案系统建设说明书-冻结定稿版.md
  GOVERNANCE-FREEZE.json

scripts/audit/
  domains/<capabilityDomain>/manifest.json
  generated/law-registry.generated.json
  reports/law-quality.report.json

.governance/runtime/
  change.json
  releases/<candidateId>/**

tests/governance/**
```

按需路径：

TXT

```
docs/blueprints/**
docs/specs/**
scripts/exam/**
scripts/dynamic-proof/**
```

规则固定：

1. READ_ONLY 不创建任何路径。
2. .governance/runtime/** 和 scripts/audit/reports/** 必须 gitignore，不得提交。
3. scripts/audit/generated/** 精确只有 law-registry.generated.json，只能由第 68 节固定的 registry 构建命令写入。
4. CI 报告写入仓库外 ephemeral path。
5. Manifest 只登记 domain 与路径主权，不要求 inspector、predicates 或逐法条文件。
6. Exam、Dynamic Proof、blueprint 和 spec 只有 active law 的 evidenceModes 或实际合同要求时才建立。
7. 不得以 v2/new/legacy/manual/compat/shadow 后缀建立长期同职路径；第 43.2 节迁移路径必须在删除日期前退出。
8. PDF 只有 freeze manifest 明确启用时才生成；PDF 不取得权威主权。

9 parser、compiler、quality 和测试的内部目录、文件名及实现技术由仓库决定，不属于本说明书冻结内容。

第三部分 | 冻结文件与两份 active 法案正文

7. 本说明书的冻结机制

docs/governance/GOVERNANCE-FREEZE.json 顶层只允许：

JSON

```
{
  "schemaVersion": 2,
  "status": "SPEC_FROZEN",
  "specPath": "docs/governance/法案系统建设说明书-冻结定稿版.md",
  "specSha256": "<64 lowercase hex>",
  "freezeVersion": "FROZEN-FINAL-V3.2",
  "amendmentAuthority": "explicit-human-only",
  "derivatives": {
    "pdf": {
      "enabled": false,
      "path": null,
      "sourceSha256": null,
      "fileSha256": null
    }
  }
}
```

冻结规则：

1. specSha256 对本说明书原始 UTF-8 字节计算 SHA-256；文件不得自写自身 hash。
2. SPEC_FROZEN 只锁定本说明书及 manifest 明确启用的派生物，不锁定 AGENTS、法案、registry 或实现，也不说明它们已经通过。
3. 第 68 节固定的治理入口在 CHANGE、READY、RELEASE 开始前核对说明书 freeze，并检查当前 AGENTS 是否符合第 45 节；READ_ONLY 直接读取并报告事实。
4. SYSTEM_PROVEN 与一个 candidateId 和 RELEASE context 绑定；任何 candidate 变化立即失效。
5. SYSTEM_PROVEN 不写回 freeze manifest，不反向取得规范主权；需要留档时只导出不可变 proof package。
6. PDF 默认关闭。开启时只作为 Markdown 的非权威派生物并绑定 source hash；PDF 不存在或未更新不得阻断默认关闭的冻结交付。
7. Markdown 标题、文件名或版本字符串不得单独宣称 SPEC_FROZEN；只有 freeze manifest 的当前有效 hash 可以证明冻结成立。

derivatives.pdf.enabled=false 时 path、sourceSha256、fileSha256 必须全部为 null；为 true 时三者必须非空，sourceSha256 必须等于 specSha256，fileSha256 绑定 path 指向的 PDF 原始字节。PDF 始终不取得权威主权。

8. active 法案正文精确为两份

人工维护并拥有 active law corpus 主权的文件精确为：

TXT

```
.github/instructions/agent-rules/基础法案.md
.github/instructions/agent-rules/功能法案.md
```

只有这两份文件中的 law-record fence 会被 active law parser 读取并进入 registry。

8.1 本说明书与 active law 的边界

本说明书规定法案系统的上位结构；第 66 节直接规定项目根治理规则。blank bootstrap 与 existing migration 按第 13、14 节把适用的长期稳定治理规则写入基础法案。

本说明书不保存 law-template fence，不作为运行时 active law source，不为每条根规则生成独立 spec、anchor、predicate、Exam 或 mutant。

第 13 节只允许一个 law-record-example fence；它是结构示意，不进入 registry。

允许承接或投影法案、但不得创造新义务的对象只有：

TXT

```
受本说明书约束的 AGENTS.md；
按需 blueprint 与 spec；
Manifest；
唯一 law registry；
CHANGE 范围记录与 RELEASE 候选绑定；
一份逐 lawId 质量报告。
```

8.2 两份源头的闭环

防止第三法案不依赖全仓关键词扫描。active-law parser 只读取第 8 节固定的两份 active law；唯一 registry builder 只把这两份法案写入唯一 registry。其它 instruction、README、AGENTS、施工图、规格、注释、任务上下文和 generated artifact 均不能成为 active law source，也不得覆盖两份 active law。parser、builder 的内部文件名、实现语言和模块拆分不属于冻结内容。

本说明书的 law-record-example 只用于定义 parser 合同，不进入 active law corpus。CHANGE 范围记录与 RELEASE 候选绑定只记录机器字段，不取得法案主权。第 46 节只校验 AGENTS 与本说明书的符合性，不通过 freeze manifest 冻结 AGENTS 字节。

9. 两份法案的固定 front matter

《基础法案.md》开头精确使用：

YAML

```yaml
---
documentType: base-law
schemaVersion: 2
status: active
authorityLevel: 100
containerOwner: law-system
lawIdPrefix: LAW
---
```

《功能法案.md》开头精确使用：

YAML

```yaml
---
documentType: functional-law
schemaVersion: 2
status: active
authorityLevel: 90
containerOwner: law-system
lawIdPrefix: LAW
---
```

任何字段缺失、额外字段、值不一致、重复 key、重复 front matter 或第二元数据区均必须 FAIL。

10. 基础法案容纳长期稳定的根治理与元治理规则

基础法案的物理结构：

MD

```
# 基础法案

## capabilityDomain: <stable-governance-domain>

### [LAW-<STABLE-GOVERNANCE-DOMAIN>-001-V1] ...
```

固定边界：

1. 基础法案只容纳不随普通业务迭代频繁变化、跨版本长期适用的治理 domain。
2. 第 66 节项目根规则必须完整出现。
3. 根规则覆盖权威、唯一真相与 Owner、任务范围、交付等级、迁移、证据、冻结和发布。
4. parser 对基础法案与功能法案使用相同算法。
5. 新增长期稳定治理规则必须对应不能由现役规则覆盖的独立风险，并获得人类明确授权。

基础法案不得包含产品流程、业务字段、页面、API、表、provider、业务状态机或单次任务范围。稳定治理法条可以约束 parser、compiler、质检和代码纯洁度的不变量，但不得把具体实现步骤、测试用例或报告格式写成法条。

11. 功能法案容纳随业务与能力持续迭代的法条

功能法案按 capabilityDomain 组织：

```MD
# 功能法案

## capabilityDomain: <domain>

### [LAW-<DOMAIN>-001-V1] <标题>
```

固定规则：

1. 不再强制建立 law-system 和 audit-system 固定 meta corpus。
2. 业务 domain 与确有必要的治理 domain 按 capabilityDomain 字典序排列。
3. 一个区块只容纳同名 domain 的 lawId。
4. 区块内按 lawKey 编号与版本升序排列。
5. 物理文件完整性 Owner 是 law-system；每个 block 的逻辑 Owner 是 metadata.capabilityDomain。
6. 修改一个 domain 不得格式化或整理其它区块。
7. parser、registry、scope、quality 的实现细节由普通测试约束，不得递归立法为固定法条工厂。

第四部分 | lawId 与法条原子语法

12. lawId 固定格式、lawKey 与历史谱系

lawId 正则固定为：

REGEX

```
^LAW-[A-Z0-9]+(?:-[A-Z0-9]+)*-[0-9]{3}-V[1-9][0-9]*$
```

映射规则：

TXT

```
capabilityDomain = lower-kebab-case
lawId domain segment = capabilityDomain 转大写
lawKey = 删除末尾 -V<major> 的 lawId
```

示例：

TXT

```
capabilityDomain: account-person-workspace
lawKey: LAW-ACCOUNT-PERSON-WORKSPACE-001
lawId: LAW-ACCOUNT-PERSON-WORKSPACE-001-V1
```

编号与版本规则：

1. 每个 domain 从 001 单调递增。
2. V1 的 supersedesLawId 必须为 null。
3. Vn（n>1）必须精确 supersede 同 lawKey 的 V(n-1)，不得跳号。
4. requirement、forbidden、appliesTo、Owner、evidenceModes、failure 条件或规范语义任一变化，major 必须加一。
5. 新 major 可以在第 43.2 节兼容窗口内与旧 major 并存；两者适用范围、消费者和删除日期必须明确。
6. 兼容窗口结束后当前工作树只保留 active 最新版本；历史只在 Git 或 HISTORY_ONLY 路径。
7. 无关语义必须申请新编号，不得复用旧 lawKey。

版本关系只按当前 active law corpus 和 supersedesLawId 裁决。前代退役后不要求继续存在于 active corpus；parser 只验证 supersedesLawId 与当前 lawId 具有相同 lawKey，且 major 精确为当前 major 减一，不为此建立历史 registry。Git 历史只用于人工追溯，不进入普通 quality，也不得成为当前主权或 PASS 前提。

13. 每条 active 法条的精确 Markdown 结构

下面使用 law-record-example 只为避免本说明书自身被 active parser 误识别。active 两份法案中的实际 fence label 必须是 law-record：

`LAW-RECORD-EXAMPLE

````
### [LAW-<DOMAIN>-001-V1] <中文标题>

```law-record
{ ...canonical law JSON... }
````
````

`

active-law parser 必须把上述内层 `law-record` 视为被外层示例 fence 包裹的数据，不得把它计入 active block；active parser 只接受两份法案中未嵌套、未缩进、行内容精确等于三个反引号加 `law-record` 的 fence。

法条 section 不得增加背景故事、当前实现盘点、开发日志、TODO、方案比较或未进入 metadata 的隐藏要求。law-record 只规定有期限迁移义务；具体期限、日期和删 spec。实现落位进入按需 blueprint，机器合同进入按需 spec，CHANGE 范围进入 runtime change record。

14. law-record JSON 精确字段

每个 block 只允许：

```json
{
  "schemaVersion": 2,
  "lawId": "LAW-<DOMAIN>-001-V1",
  "supersedesLawId": null,
  "status": "active",
  "tier": "P0",
  "capabilityDomain": "<lower-kebab-case>",
  "enforcementMode": "static",
  "evidenceModes": ["static.ast"],
  "summary": "必须……",
  "appliesTo": ["src/<domain>/**"],
  "excludes": [],
  "forbidden": ["不得……"],
  "dependsOnLawIds": []
}
```

字段约束：

14.1 schemaVersion、status、supersedesLawId

TXT

schemaVersion 固定整数 2；
status 固定 active；
V1 supersedesLawId=null；
Vn 精确指向同 lawKey 的 V(n-1)；前代已退役时无需保留其 active block，亦不得为验证版本关系建立历史 registry。

14.2 tier

只允许 P0、P1、P2；全部都是失败关闭，不存在 warning law。

14.3 enforcementMode 与 evidenceModes

enforcementMode 只允许 static、exam、dynamic、composite。

TXT

只有 static.* -> static
只有 exam.* -> exam
只有 dynamic.* -> dynamic
存在两类或三类 -> composite

evidenceModes 只允许：

TXT

```
static.filesystem
static.text
static.ast
static.types
static.contract
static.dependency
static.config
static.json
static.yaml
static.sql
static.migration
exam.golden
exam.property
exam.state-machine
dynamic.browser
dynamic.service
dynamic.database
dynamic.storage
dynamic.provider
dynamic.authorization
dynamic.recovery
dynamic.performance
```

数组非空、无重复、按上表排序。

14.4 summary

TXT

```
以“必须”或“只能”开头；
只描述一个可独立裁决的不变量；
12 至 240 个 Unicode code point；
不得出现建议、尽量、可能、一般、视情况或原则上；
不得写开发日志、文件清单或单次任务范围；有期限迁移的机器字段进入第 69 节 migration contract，不塞入法条 summary。
```

14.5 appliesTo 与 excludes

TXT

```
仓库相对 POSIX glob；有效 UTF-8 NFC；不得绝对路径、..、反斜杠、负向 glob、brace、extglob、字符组或问号；
只允许精确文件、目录 /**、目录 /**/*.<literal-extension>；
数组字典序、无重复、无包含冗余；
appliesTo 非空；
只有 project-governance、law-system、audit-system 元法条允许根级 **/*；
excludes 只能排除 appliesTo 内明确子面，不得排除生产主链逃避执法。
```

14.6 forbidden

1 至 16 项；每项以“不得”或“禁止”开头；一项一个违法条件；不得以模糊阶段词豁免现役义务。有期限迁移的法条写明不得无期限并由关联 migration spec 提供第 43.2、69 节机器字段，不视为豁免。

14.7 dependsOnLawIds

只能引用当前 active lawId；按 lawId 排序且无重复；不得自引；依赖图无环；跨域依赖必须进入 registry，并在存在施工图时同时反映于相关施工图；不得分散 Owner。

15. 法条正文只写一次

law-record 是法条唯一正文。标题只提供阅读导航，不重复 summary、forbidden、evidenceModes 或违法后果。parser 验证标题 lawId、domain、版本关系和 JSON 字段，不维护第二份逐字镜像。

16. tier 违法后果与 parser 算法

tier 的违法后果固定为：

TXT

P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。
P1：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P1 finding；对应质量入口必须非零退出，并阻断合并和发布。
P2：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P2 finding；对应质量入口必须非零退出，并阻断合并。

16.1 law-parser 的精确解析算法

TXT

1. 读取原始字节；拒绝 BOM、非 UTF-8、非 NFC 路径和 CRLF；
2. 验证 front matter 从首字节开始，以精确 ---\n 打开和关闭；
3. 唯一 yaml parser 解析，禁止重复 key；
4. 逐行维护 Markdown fence 状态；
5. 只有精确行 ```law-record 打开法条 block；只有精确行 ``` 关闭；
6. 禁止未关闭、嵌套、缩进 fence 和 block 内注释；
7. block 必须是单一 JSON object，禁止未知字段；
8. 找到最近 level-3 law heading，一个 heading 精确一个 block；
9. 找到最近 level-2 capabilityDomain heading；基础法案和功能法案算法相同；
10. 验证标题 lawId、domain、major 与 supersedes；
11. 记录原始 lineStart/lineEnd、metadata hash、section hash；
12. EOF 后拒绝孤立 law heading 或 law-record；
13. 任一解析错误只产生结构冲突，不生成部分 registry。

16.2 其它 parser

Manifest、blueprint、spec 的 parser 必须拒绝重复
key、未知字段、空字符串、非法路径、未替换占位符和默认补值。其内部文件名、实现语言和模块拆分由仓库决定。

第五部分 | capabilityDomain、Owner 与 Manifest

17. capabilityDomain 命名与创建

capabilityDomain 正则固定为：

REGEX

```
^[a-z0-9]+(?:-[a-z0-9]+)*$
```

每个 capabilityDomain 必须代表一个能够独立回答下列问题的完整责任域：

TXT

```
它拥有哪一种事实或能力；
它创建什么；
它读取什么；
它修改什么；
它裁决什么；
它明确不得接管什么。
```

不得创建以下 domain：

TXT

```
common、shared、utils、misc、core2、platform2、general、helpers、temporary、legacy、compat。
```

新 capabilityDomain 只有在用户明确授权新增能力域，并且现有 domain 无法合法承接时才允许创建。Codex 不得为了整理代码、追求优雅或减少文件行数自行新增 domain。

18. 物理容器 Owner 与逻辑 law Owner 分离

固定关系：

TXT

```
基础法案.md / 功能法案.md 的文件完整性 Owner = law-system
每个 law-record block 的逻辑 Owner = metadata.capabilityDomain
```

因此：

TXT

```
一个物理文件可以容纳多个逻辑 Owner；
一个 lawId 只能有一个逻辑 Owner；
Owner 不得通过拥有法案文件路径接管其它 lawId；
Audit 必须按 lawId block 定位主权，不得按 Markdown 文件整体绑定。
```

19. Manifest 固定路径

每个 active capabilityDomain 精确存在：

TXT

```
scripts/audit/domains/<capabilityDomain>/manifest.json
```

目录名与 manifest.capabilityDomain 必须相同。Manifest 是路径主权登记，不是第二法案、测试计划或报告配置。

普通治理测试可以跨 domain 组织；不得要求每个 domain 固定建立同职 inspector 或 predicate 模块。

20. Manifest 只登记 Owner 与路径

每个 capabilityDomain 的 Manifest 顶层只允许：

JSON

```
{
  "schemaVersion": 3,
  "capabilityDomain": "<lower-kebab-case>",
  "status": "active",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

规则固定：

1. lawIds 精确等于该 domain 当前 active lawId。
2. ownedPaths 表示唯一 primary 写主权；observedPaths 只读。
3. Manifest 不登记逐法条 blueprint、spec、predicate、Exam、Dynamic Proof、report sink 或 mutant。
4. Manifest 不得写 PASS 条件、例外或新义务。
5. 路径只允许精确文件或目录 /**；primary Owner 重叠必须 FAIL。
6. lawIds 按 lawId、两个路径数组按 UTF-8 path bytes 升序排列；各数组无重复，同一精确路径不得同时出现在 ownedPaths 与 observedPaths。

证据绑定由 law-record 的 evidenceModes
与普通治理测试共同裁决；只有确有机器合同或外部运行义务的法条，才按第 24、60、61 节建立附加对象。

21. 候选文件角色与 ownership

候选路径只分为：

TXT

```
GOVERNANCE_AUTHORITY；
GENERATED_REGISTRY；
QUALITY_REPORT；
RUNTIME_IGNORED；
TEST_ONLY；
HISTORY_ONLY；
PRODUCTION。
```

规则固定：

1. 本说明书、freeze、AGENTS、两份 active law、Manifest 及按需 blueprint/spec 为 GOVERNANCE_AUTHORITY。
2. 第 26 节唯一 registry 为 GENERATED_REGISTRY；第 57 节报告为 QUALITY_REPORT；.governance/runtime/** 为 RUNTIME_IGNORED。
3. tests/** 及未被生产消费的 fixture/example 为 TEST_ONLY；明确 docs/history/** 为 HISTORY_ONLY；其余候选路径默认为 PRODUCTION。
4. PRESENT 的 PRODUCTION 文件必须命中一个且仅一个 Manifest ownedPaths。
5. 非生产路径被生产 import、发布包、配置或状态写入消费时，按 PRODUCTION 裁决。
6. DELETED 不读取工作树字节；删除权限按 baseSha、allowedDeletePaths 和原 Owner 验证。
7. rename 视为旧 path DELETE 与新 path PRESENT。
8. symlink 不得越出仓库根；submodule、unmerged index 和特殊设备文件必须阻断。

固定结果：

TXT

```
零 Owner -> UNOWNED_PRODUCTION_FILE；
多 Owner -> MULTIPLE_PRIMARY_OWNERS；
非生产角色被生产消费 -> ROLE_ESCALATION_REQUIRED；
未授权删除 -> UNAUTHORIZED_PRODUCTION_DELETE。
```

22. 施工图按复杂度建立

SYSTEM-BLUEPRINT.md 与 capabilityDomain blueprint 不再是每个仓、每个 domain、每条法案的强制配件。

出现以下任一情形时，才必须建立机器可读施工图：

TXT

```
三个以上 domain 的依赖拓扑；
跨域合同；
跨仓合同；
多状态恢复流程；
无法从代码与 spec 直接确定的 Owner 或调用方向。
```

施工图的最小机器 block：

JSON

```
{
  "schemaVersion": 1,
  "blueprintId": "<stable-id>",
  "status": "active",
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "contractRefs": [],
  "dependencyEdges": [
   {
    "from": {"repo": "local", "domain": "<capabilityDomain>"},
    "to": {"repo": "local", "domain": "<capabilityDomain>"},
    "contractRef": null
   }
  ],
  "prohibitedDependencyEdges": [
   {
    "from": {"repo": "local", "domain": "<capabilityDomain>"},
    "to": {"repo": "local", "domain": "<capabilityDomain>"},
    "lawId": "LAW-..."
   }
  ]
}
```

lawIds 与 contractRefs 是 registry 反向建立 law-to-contract 关系的唯一来源；不得依赖 Markdown 标题、行号或自然语言推断。

lawIds 按 lawId、contractRefs 按路径的 UTF-8 bytes 升序排列，均无重复。

edge endpoint 精确为 {repo, domain}。repo 只允许字面值 local 或 active cross-repo contract 中的 canonical owner/repo；domain 必须是对应仓的现役 capabilityDomain。from 表示依赖方或调用方，to 表示被依赖方或提供方；两端不得相同。

dependencyEdges[] item 只允许 from、to、contractRef。contractRef 只允许 null 或 top-level contractRefs 中的 active spec 路径；跨域或跨仓合同存在时不得为 null。

prohibitedDependencyEdges[] item 只允许 from、to、lawId。parser 只验证 lawId 同时存在于 top-level lawIds 且为 active；该 law 是否禁止此方向由普通治理测试按明确合同和依赖关系裁决，不要求 parser 解释自然语言。施工图不得自行创造禁令。

两个数组分别按 from.repo、from.domain、to.repo、to.domain、最后一个字段的 UTF-8 bytes 升序排列。同一数组不得重复；相同 from/to 不得同时出现在允许与禁止数组；未知字段、非法 endpoint、悬空引用、逆序、重复或允许/禁止冲突必须 FAIL。

blueprint 可以是独立 JSON 文件，也可以是 Markdown 中唯一的 blueprint-record JSON fence；builder 只读取该机器 block，不解析正文形成隐含合同。

施工图只记录 topology、Owner、contract 和 prohibited edge；不得为每个 lawId 建立形式化 anchor。

简单 domain 可以只使用法案、Manifest、代码与测试，不得为了满足目录形式创建空 blueprint。

23. 施工图正文

施工图只写当前系统需要长期维持的结构，不设固定十五章，不要求空章节。

正文应覆盖实际存在的：

TXT

```
终态对象；
Owner；
输入输出；
持久化与状态；
调用方向；
错误恢复；
安全边界；
验收条件。
```

不存在的对象不得以占位章节、TODO 或未来规划写入。

24. 规格按机器合同建立

只有法条涉及字段、API、事件、存储、状态机、配置、安全合同或跨仓合同，且代码类型本身不足以成为唯一机器真相时，才建立 spec。

spec 必须具有：

TXT

```
specId；
version；
Owner；
适用 lawIds；
精确合同内容。
```

spec 的最小 schema：

JSON

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<stable-id>",
  "status": "active",
  "version": 1,
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "specKind": "typed-contract",
  "contract": {}
}
```

lawIds 按 lawId 排序且无重复。

specKind 只允许 typed-contract、api、event、storage、state-machine、config、security、migration、cross-repo-contract、contract-pin。specKind 唯一决定 contract 使用的 schema；同一 specKind 和 schemaVersion 不得同时对应两套 schema，也不得由文件路径、Owner 或运行参数改选 schema。spec parser 校验公共外壳后，按 specKind 进入唯一合同 schema；schema 的内部实现位置和技术由仓库决定。公共外壳未知字段、缺失 Owner、空 lawIds、未知 specKind、合同不符合对应 schema 或未被 active law 引用必须 FAIL。

version 必须是正整数；contract 的规范内容发生变化时递增，旧版本只可在第 43.2 节兼容窗口内继续存在。spec 不得重复法条正文，不得为每条 lawId 强制建立一份文件，不得用空数组和模板字段制造形式完整。

跨仓合同使用 provider descriptor 与 consumer pin。破坏性改变提升 version；消费者在声明的兼容窗口内独立部署，不要求同一任务同步完成全部仓库 RELEASE。

25. 法案、Manifest、施工图、规格与任务记录的分工

TXT

```
法案：规定必须、禁止、Owner、证据等级与违法后果；
Manifest：登记 domain、lawIds 与路径主权；
施工图：按需记录复杂拓扑和调用边界；
spec：按需记录代码之外的机器合同；
change record：记录本次允许写入、删除的路径和 baseSha；
registry：投影当前 active laws、Owner 和可选合同引用；
质量报告：逐 lawId 记录当前候选的裁决与证据。
```

任何对象不得重复取得另一对象的主权。

第六部分 | 唯一 law registry 与 canonical 生成

26. 唯一 registry

唯一机器索引固定为：

TXT

```
scripts/audit/generated/law-registry.generated.json
```

不得建立 law index、authority census、enforcement bindings、audit plan、inspector registry 或其它平行 generated projection。

registry 只投影当前 active law、Owner、source location、evidenceModes 和按需 contract refs；不拥有规范主权或 PASS 权。

27. registry 顶层 schema

JSON

```
{
  "schemaVersion": 3,
  "sourceSpec": {"path": "...", "sha256": "..."},
  "lawCorpus": {
    "baseLawPath": "...",
    "baseLawSha256": "...",
    "functionalLawPath": "...",
    "functionalLawSha256": "..."
  },
  "laws": [],
  "domains": []
}
```

domains[] 每项只允许：

JSON

```
{
  "capabilityDomain": "...",
  "manifestPath": "...",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

全部数组按稳定 key 排序；禁止时间、机器名、用户名、绝对路径和随机值。

28. registry law record

JSON

```
{
  "schemaVersion": 2,
  "lawId": "LAW-...",
  "status": "active",
  "supersedesLawId": null,
  "tier": "P0",
  "capabilityDomain": "...",
  "enforcementMode": "static",
  "summary": "...",
  "appliesTo": [],
  "excludes": [],
  "forbidden": [],
  "evidenceModes": [],
  "dependsOnLawIds": [],
  "source": {
    "path": "...",
    "lineStart": 1,
    "lineEnd": 10,
    "sectionSha256": "..."
  },
  "contractRefs": []
}
```

contractRefs 只收录 lawIds 包含该法条的 active blueprint/spec，并精确使用：

JSON

```
{"path": "repo/relative/path", "sha256": "<64 lowercase hex>"}
```

数组按 path 的 UTF-8 bytes 升序排列，sha256 绑定对应 artifact 原始字节；没有按需合同的法条必须为空数组。registry law record 必须忠实投影 law-record 的全部规范字段，不收录逐法条 predicate、Exam、mutant 或 report sink。

29. hash

文件 hash 使用原始 UTF-8 字节；law section hash 使用法条完整原始字节；registry hash 使用第 30 节 canonical JSON。

Git object id 使用当前仓库 object format 的小写十六进制，不固定为 40 位；同一候选中的 Git id 必须使用同一 object format。

不得把时间、路径之外的机器环境、报告或运行结果写入 source hash。

30. canonical JSON

对象 key 按 Unicode code point 升序递归排序；数组保持其所属 schema 规定的顺序。registry 的 laws 按 lawId、domains 按 capabilityDomain、contractRefs 按 path 排序；字符串数组按各自字段规则排序。缩进 2；单 LF 结尾；禁止注释、尾空格、NaN、Infinity、undefined 和时间字段。

31. 唯一 registry builder

仓库只允许一个承担 registry 生成主权的 builder，并由第 68 节固定的 audit:build 调用。builder 的内部文件名、实现语言和模块拆分由仓库决定。

输入只包括：

TXT

```
说明书与 freeze manifest；
当前 AGENTS；
两份 active 法案；
全部 active Manifests；
固定按需目录中全部 status=active 的 blueprints/specs。
```

唯一输出是 law-registry.generated.json 的 canonical bytes。builder
不读取生产候选、历史报告、网络、数据库、浏览器、墙钟或随机源，不输出 verdict。

32. source-integrity preflight

builder、projection check 与 READY、RELEASE quality 共用以下只读检查：

TXT

```
说明书 freeze；
AGENTS 是否符合第 45 节；
两份 active law 结构；
active law source 精确为固定两份；
Manifest Owner 与路径重叠；
按需 blueprint/spec 的 schema、lawIds 与 Owner 完整性；
path encoding 与 case collision。
```

CHANGE 只验证说明书 freeze、AGENTS 边界及本次实际触碰的治理 source；不遍历全量 registry、ownership 或未受影响合同。READY、RELEASE 执行上述完整 preflight。freeze 不一致时不得宣称 SPEC_FROZEN。

失败只输出确定性 code，不写 registry 或报告。

33. registry 构建

audit:build 在内存生成唯一 registry，验证可重新解析后以临时文件和原子 rename
替换目标文件。失败时保留旧文件，不得留下半写状态。

audit:build 不写质量报告，不自动补法案、Manifest、blueprint 或 spec。

34. projection check

audit:projection:check 重新调用同一 builder，比对唯一 registry exact bytes，并证明检查前后候选未变化。

registry 缺失返回 GENERATED_FILESET_INVALID；字节陈旧或被手改返回
GENERATED_PROJECTION_STALE。检查不得自动调用 audit:build 修复。

第七部分 | 任务范围与交付等级

35. 任务按交付等级分级

任务只分为 READ_ONLY、CHANGE、READY、RELEASE 四级。不得以较低等级冒充较高等级。

分类顺序固定：人类明确禁止写入时为 READ_ONLY；明确要求生产发布、最终发布证明或 SYSTEM_PROVEN 时为 RELEASE；明确要求合并、部署前就绪或 READY 验收时为 READY；其它包含仓库写入的任务为 CHANGE。执行者不得自行降级；人类未要求更高等级时不得自行升级。

35.1 READ_ONLY

READ_ONLY 直接读取权威文件和目标文件并回答，不建立任务事务。

READ_ONLY 不加锁、不建立任务事务或任务 artifact，也不写入 .governance/runtime/**。

只读结论不得宣称代码已经修改、候选已经验证或系统已经发布。

35.2 CHANGE

CHANGE 允许在明确路径内修改。首个写入前只建立第 36 节 change record，不加锁，不生成其它任务事务 artifact。

CHANGE 只证明本次差异未越界并通过适用的普通测试；不得据此宣称 READY、RELEASE 或 SYSTEM_PROVEN。

35.3 READY

READY 表示当前候选已经完成合并或部署前的静态检查、普通测试和适用验收，但尚未取得最终发布证明。

READY 不得替代真实生产环境、外部依赖或动态运行证据。

35.4 RELEASE

RELEASE 是唯一启用完整候选绑定、全量质量报告、适用 Dynamic Proof 和发布证明包的等级。

只有 RELEASE 可以产生 SYSTEM_PROVEN。

36. CHANGE 的唯一范围记录

CHANGE 首次写入前只允许建立一份 gitignored record：

JSON

```
{
  "baseSha": "<git object id>",
  "allowedWritePaths": [],
  "allowedDeletePaths": []
}
```

规则固定：

1. baseSha 是开始修改时的 Git HEAD。
2. 两个路径数组只能包含仓库相对 POSIX 精确路径或目录 /**，无重复、无重叠。
3. record 只允许三个法定字段，不得保存请求副本、任务摘要、闭包、路由、上下文、回执或证据。
4. record 不构成规范主权，不得提交。
5. 修改范围扩大时，必须在首次触碰新增路径前更新同一 record；不得创建第二份 record。

6 首次写入前必须检查目标路径的 Git 状态；目标已有用户改动且人类未明确授权在其上继续编辑时，立即停止。三字段 record 只证明路径范围，不得声称能够证明改动作者或自动恢复用户字节。

37. CHANGE 的范围判定

允许修改的路径仅来自：

TXT

```
人类明确点名的路径；
完成该修改不可分割的同一合同路径；
由唯一生成器确定发生变化的派生路径。
```

第二、三项只有在任务确实需要时才加入；不得预授权全仓，不得以整理、抽象、未来复用或顺手修复扩大范围。

READ_ONLY 不执行本节。

38. 范围扩张

范围扩张只允许生成物、同一 Owner、同一合同、适用证明或旧路径退休的直接闭包。

扩张前必须把新增路径写入同一 change record。record 仍只保存 baseSha、allowedWritePaths 和 allowedDeletePaths，不建立其它范围事务对象。

新增 API、CLI、事件、队列、表、列、环境变量、依赖、监听器、外连主机、浏览器权限或跨仓合同，仍须有人类明确授权或现役法案的直接依据。

39. 法案与既有治理迁移

修改本说明书必须有人类明确授权 source-spec-amendment；授权只覆盖本说明书及完成冻结所必需的 freeze manifest，不自动授权修改 AGENTS、active law、registry 或实现。正文定稿后才计算并写入新的 specSha256。

39.1 法案修订

修改 active 法案必须有人类明确授权 law-amendment。

修订可以在分支、工作树或隔离目录中验证；不得要求所有关联实现必须在同一任务达到 RELEASE。修订后的 active law、registry 和实现只有在各自完成时才取得对应状态，未完成项必须如实标记；law-amendment 不改变本说明书的 freeze。

39.2 既有治理迁移

既有治理允许分步迁移。每一步必须声明当前权威、目标权威、受影响路径、验证方式和剩余工作；不得把中间状态冒充 READY 或 RELEASE。

迁移期间允许第 43.2 节规定的有期限兼容机制。

40. RELEASE 候选绑定

进入 RELEASE 时必须固定：

TXT

```
candidateId；
Git commit；
法案与 registry hash；
适用质量报告 hash；
适用 Dynamic Proof hash；
跨仓合同版本与参与仓发布结果。
```

完整绑定由 quality:check 的 RELEASE 分支在最终报告通过后写入 .governance/runtime/releases/<candidateId>/**。第 B.9 节的 proof package 就是唯一 RELEASE 绑定 artifact，不得另建 release receipt 或同义绑定层。绑定缺失或 candidate 漂移时，RELEASE 失败。CHANGE 与 READY 不生成该完整绑定。

41. 固定阻断码

任务控制只允许以下阻断码：

TXT

```
FREEZE_HASH_MISMATCH
ENTRY_CONTRACT_MISMATCH
USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW
CHANGE_RECORD_MISSING
CHANGE_RECORD_INVALID
BASE_SHA_MISMATCH
SCOPE_DIFF_OUTSIDE_ALLOWED_PATHS
DIRTY_TARGET_NOT_AUTHORIZED
LAW_AMENDMENT_NOT_AUTHORIZED
SOURCE_SPEC_AMENDMENT_NOT_AUTHORIZED
RELEASE_CANDIDATE_UNBOUND
RELEASE_EVIDENCE_MISSING
CANDIDATE_DRIFT
```

READ_ONLY 只在目标或权威不可读取时停止；不得因为缺少 runtime task artifact 而停止。

42. Codex 范围禁区

未经明确授权，Codex 不得：

TXT

```
修改 change record 之外的路径；
新增产品范围、公开接口、持久化结构或外部权限；
建立第二法案、第二 registry、第二 Owner、第二 compiler 或第二质量入口；
用 fixture、mock、no-op 或固定返回冒充生产能力；
用 CHANGE 或 READY 结论冒充 RELEASE；
覆盖、删除、恢复或移动用户原有改动。
```

迁移桥、用于迁移的 feature flag、dual-read、dual-write 和 shadow traffic 依第 43.2 节裁决，不作绝对禁止。长期产品 feature flag 由对应业务 law 管理，不得用于暗中保留旧真相源。

43. 交付与迁移

43.1 渐进建设

允许按可验证的小步建设、MVP、preview 或阶段目标交付。每一步必须明确实际完成内容、未完成内容和当前等级。

阶段成果不得被写成完整生产终态，不得把未实现能力记为 PASS。

43.2 有期限兼容机制

兼容桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic 或临时 adapter 只有同时满足以下条件才合法：

TXT

```
唯一 Owner；
旧、新路径身份明确；
开始日期；
监控指标；
回滚条件；
最长存续期限；
删除日期；
删除验收标准。
```

兼容机制不得形成两个长期真相源；到期未删除必须 FAIL。

43.3 跨仓交付

跨仓通过版本化合同独立部署。合同内容变化递增 version；破坏性改变同时声明兼容窗口和消费者迁移期限。

一仓 CHANGE 或 READY 不得冒充另一仓完成；只有整体 RELEASE 声明才要求全部必要参与仓的发布结果。

44. 最终 scope check

CHANGE、READY、RELEASE 均以 baseSha 到当前候选的实际 Git 差异为分母。

governance:scope:check 必须验证：

TXT

```
实际 WRITE 全部属于 allowedWritePaths；
实际 DELETE 全部属于 allowedDeletePaths；
首次写入前的 dirty target 已获得明确授权，否则未被触碰；
法案修改具有对应人类授权；
generated bytes 与唯一生成器对同一输入的输出完全一致；
新增生产路径具有唯一 Owner；
新增外部 surface 具有现役依据。
```

任一不成立，返回第 41 节对应阻断码并非零退出。

45. AGENTS.md 的边界

AGENTS 只写入口、权威顺序、READ_ONLY 零写入、CHANGE 范围记录、交付等级、RELEASE 候选绑定、有期限迁移、active law 修改授权和失败关闭；不得复制业务法案、手工路由表或完整质量流程。

46. 治理入口

READ_ONLY 可以直接读取，不强制运行 entry。

CHANGE 首次写入前运行 governance:entry，只检查说明书 freeze、AGENTS 边界、唯一质量入口和目标路径初始 Git 状态。READY、RELEASE 额外检查法案与 registry。未获授权的 dirty target 返回 DIRTY_TARGET_NOT_AUTHORIZED。entry 只读，不创建 task artifact，不输出 PASS 文件。

47. 读取规则

TXT

```
READ_ONLY：读取回答问题所需的权威和目标文件，不落盘；
CHANGE：读取相关法案、registry 和目标文件；
READY：在 CHANGE 基础上读取适用验收合同；
RELEASE：读取完整候选、全部 active laws、registry 和发布证据。
```

说明书 freeze 不匹配或 AGENTS 违反第 45 节时，CHANGE、READY、RELEASE 停止；READ_ONLY 仍可读取并报告事实。

第八部分 | 质量与证据

48. Static Audit 的边界

Static Audit 只证明当前候选的文件、文本、AST、类型、schema、合同、依赖、配置和迁移声明。

Static Audit 不得启动外部系统，不得把测试夹具、历史报告或截图冒充当前运行证据，不得输出 RELEASE verdict。

49. CandidateIdentity

CHANGE 与 READY 以 baseSha 到当前工作树的实际 Git 差异为候选分母。

RELEASE 只允许绑定一个已提交的 Git tree；受治理的生产文件不得存在未提交或未跟踪变化。candidateId 依第 30 节对以下对象的 canonical JSON bytes 计算 SHA-256：

JSON

```
{
  "gitCommit": "<git object id>",
  "baseLawSha256": "<64 lowercase hex>",
  "functionalLawSha256": "<64 lowercase hex>",
  "registrySha256": "<64 lowercase hex>"
}
```

Git commit 已绑定该提交的完整 tracked tree，禁止再维护第二份 path 清单。Dynamic Proof、质量报告和 proof package 分别以第 B.9 节 hash 绑定，不进入 candidateId。绝对路径、时间、机器名、用户名和报告内容不得进入 candidateId。

50. candidate drift

质量检查开始后，候选发生变化时：

TXT

```
当前检查失败；
本次临时结果作废；
不得发布质量报告；
RELEASE 必须重新绑定 candidateId。
```

CHANGE 与 READY 不需要在每个测试步骤生成独立 candidate receipt。

51. 一套普通治理测试

治理测试可以按 domain 或技术类型拆分文件，由仓库现有测试框架组织。只有 canonical quality 入口可以汇总治理结论；测试文件数量和目录结构不构成第二质量入口。

测试至少覆盖：

TXT

```
法案 parser；
lawId 与当前版本关系；
active law source allowlist；
Manifest Owner；
registry deterministic bytes；
scope；
证据分层；
禁止伪 PASS。
```

一条测试可以覆盖多个相关 lawId；一个 lawId 也可以由多条普通测试共同证明。不得强制一 law 一 predicate、一 law 一 Exam 或每个测试一 mutant。

52. 静态证据分母

静态证据分母由 active law appliesTo/excludes、Manifest paths、实际 contract refs 和固定治理源码求并集。

discovery、read、decode 或 parse 失败必须 FAIL。

以下路径不得作为业务实现已经成立的充分证据：

TXT

```
tests/**
fixtures/**
examples/**
artifacts/**
coverage/**
docs/history/**
scripts/audit/reports/**
```

53. 普通测试结果

普通测试使用仓库现有测试框架。作为法条证据时，必须能够定位 testId、适用 lawIds、结果和证据；具体结果格式由仓库实现。任一 finding 必须失败，不得把 warning 或覆盖率冒充 PASS。

54. 逐 lawId 裁决

READY 与 RELEASE 根据 active law、适用测试、静态检查和当前交付等级，对每个 lawId 产生一个最终裁决。CHANGE 不生成全量逐 lawId 裁决。

law 裁决只允许：

TXT

```
PASS：本等级要求的证据完整且无 finding；
FAIL：要求不成立、禁止条件出现或本等级适用证据失败。
```

每个 law 同时列出 applicableEvidenceModes 与 deferredEvidenceModes。deferredEvidenceModes 只表示该证据不属于当前等级，不能冒充已经取得；它不阻断当前等级的 PASS。RELEASE 时该数组必须为空。

55. evidenceRef

evidenceRef 必须能够定位当前候选中的文件、文本范围、AST 节点、JSON pointer、依赖边、测试结果或真实动态 locator。

自然语言“见代码”“已经测试”“应该没问题”不是 evidenceRef。

56. 唯一逐法条质量报告

READY 与 RELEASE 每次只生成一份报告：

JSON

```
{
  "schemaVersion": 1,
  "level": "READY",
  "baseSha": "<git object id>",
  "candidateId": null,
  "registrySha256": "<64 lowercase hex>",
  "laws": [
    {
      "lawId": "LAW-...",
      "applicableEvidenceModes": [],
      "deferredEvidenceModes": [],
      "outcome": "PASS",
      "evidenceRefs": [],
      "findings": []
    }
  ],
  "outcome": "PASS"
}
```

level 只允许 READY、RELEASE。RELEASE 的 candidateId 必须非空且每个 law 的 deferredEvidenceModes 必须为空；READY 可以为空。

laws[] 按 lawId 排序且精确覆盖全部 active lawId；两个 evidenceMode 数组按第 14.3 节顺序且无重复。evidenceRefs[] 是可复放定位符字符串，按 UTF-8 bytes 排序且无重复。findings[] 是 ^[A-Z][A-Z0-9_]*$ 错误码，按 UTF-8 bytes 排序且无重复；PASS 时必须为空，FAIL 时必须非空。报告顶层 outcome 只有全部 law outcome 为 PASS 时才为 PASS。

报告不得再拆成平行 decision、component/law/Owner/CI receipt 或其它同义 verdict 层。

57. 报告写入

唯一 writer 只写第 56 节报告：

TXT

```
本地 READY、RELEASE：scripts/audit/reports/law-quality.report.json；
CI：仓库外 ephemeral path。
```

报告只证明对应 baseSha/candidateId/registrySha256/level，不取得法案主权，不作为下一次证据输入。

58. 分级质量流程

TXT

```
CHANGE：scope -> 受影响 project checks 与普通测试 -> PASS | FAIL；
READY：source 与 registry -> ownership -> 完整 READY checks -> 按需 Exam -> 一份逐法条报告 -> PASS | FAIL；
RELEASE：READY -> candidate binding -> 发布适用 Dynamic Proof -> candidate recheck -> 一份逐法条报告 -> proof package -> PASS | FAIL。
```

不得在各步骤之间生成可独立宣称整体通过的中间 PASS 文件。

59. 三类证据不得互相冒充

TXT

```
Static Audit 证明静态结构；
Exam 证明确定性行为；
Dynamic Proof 证明真实外部运行。
```

任何证据只能证明其实际观察到的对象。

60. Exam 的建立与适用

只有 active law 声明 exam.* evidenceMode 时才建立 Exam；law 未声明 exam.* 时不得由执行者主观追加。对应 law 与实际差异相交，并进入 READY 或 RELEASE 时，Exam 适用。

Exam 可以覆盖多个相关 lawId，不要求每条法案一份 Exam。输入、seed、超时和失败用例必须可复放。

61. Dynamic Proof 的时点

只有 active law 声明 dynamic.* evidenceMode 时才建立 Dynamic Proof；对应 law 与实际差异相交，并进入 RELEASE 时强制执行。CHANGE 与 READY 不执行 Dynamic Proof，也不得因此被阻断。

Dynamic Proof
只证明真实浏览器、服务、数据库、存储、provider、授权、恢复或性能事实。凭证不得写入仓库或报告。

不可逆动作必须有人类逐字授权。cleanup 失败即 Dynamic Proof 失败。

外部动态条件暂时不可用时：

TXT

```
CHANGE 可以完成；
READY 可以按其静态和测试条件裁决；
RELEASE 必须失败或等待。
```

62. 证据适用性

READY 与 RELEASE 的适用性只由 law 的 evidenceModes、交付等级和实际差异共同决定，并直接写入同一逐法条报告。law 的 appliesTo 扣除 excludes 后与实际差异相交，或其 active contractRef 对应的合同发生变化，即为该 law 与实际差异相交：

TXT

```
Static：声明了对应 `static.*`、law 与差异相交，且等级为 READY 或 RELEASE 时适用；
Exam：声明了对应 `exam.*`、law 与差异相交，且等级为 READY 或 RELEASE 时适用；
Dynamic：声明了对应 `dynamic.*`、law 与差异相交，且等级为 RELEASE 时适用。
```

READY 推迟的 evidenceMode 写入 deferredEvidenceModes；RELEASE 不得推迟。CHANGE 只报告受影响检查的实际结果，不建立全量 deferred 清单。不得为了取得 PASS 临时降低 evidenceModes，也不得把所有法条机械升级为 composite。

63. 唯一质量入口

仓库只保留一个 canonical quality 入口 quality:check，并接受 CHANGE、READY、RELEASE 三级参数。编程语言和内部文件组织由仓库决定。

level 必须与第 35 节分类一致。

63.1 CHANGE

只检查允许路径、实际差异，以及与本次差异相关的 project checks 和普通测试。CHANGE 不执行全仓 law history、全量 registry/ownership、全量治理测试或逐 lawId 报告。PASS 只表示本次变更在 CHANGE 等级闭合。

63.2 READY

在 CHANGE 基础上执行 source、registry、ownership、完整 READY checks、适用 Exam 和逐 lawId 报告。PASS 只表示候选达到声明的 READY 条件。

63.3 RELEASE

绑定完整 candidateId，执行全部 active laws 的发布适用证据及其中适用的 Dynamic Proof，生成第 64.2 节候选证明包。全部通过后才可产生 SYSTEM_PROVEN。

64. CI 与发布证明包

64.1 CI

CI 只能调用同一 canonical quality 入口。普通 PR 使用 CHANGE 或 READY；自动事件不得提升为 RELEASE。

CI 报告写入仓库外临时目录，不修改 source、registry、runtime 或仓库内 report cache。最终成功或失败只认质量进程退出码。

64.2 RELEASE proof package

只有 RELEASE 的全部检查和逐法条报告通过后才生成候选证明包；证明包成功写入并完成 candidate recheck 后，quality:check 才能以 RELEASE PASS 退出。证明包只包含：

TXT

```
candidateId；
Git commit；
freeze、两份 active law 与 registry hash；
逐法条质量报告 hash；
Dynamic Proof hash；
适用跨仓合同版本与 participant release hash。
```

证明包不得包含新的规范义务或自然语言 READY。candidate 变化后，旧证明包只具有历史追溯意义。

第九部分 | 根治理规则与建制

65. blank bootstrap

空白仓首次建制只建立：

TXT

```
本说明书与 freeze；
AGENTS；
基础法案与功能法案；
第 66 节项目根治理规则；
按实际生产路径建立的 Manifests；
唯一 law registry；
一套普通治理测试；
READY 与 RELEASE 使用的一份逐法条质量报告；
quality:check。
```

blueprint、spec、Exam、Dynamic Proof、迁移桥和跨仓合同只在实际需要时建立，不得预建空壳。

bootstrap 完成 CHANGE 与 READY 后可以继续分步建设；只有进入 RELEASE 时才要求完整 candidate binding 和发布适用证据。

66. 项目根治理规则

基础法案保存长期稳定、跨版本持续适用的根治理规则。不得把无独立长期风险的 parser、compiler、report writer、测试夹具或某个实现步骤分别立为元法案。

本节项目根规则共同使用：schemaVersion=2、status=active、tier=P0、capabilityDomain=project-governance、supersedesLawId=null、appliesTo=["**/*"]、excludes=[]、dependsOnLawIds=[]；enforcementMode 由第 14.3 节从本条证据推导。每条“要”正文进入 summary，“不要”中以分号分隔的句子依次进入 forbidden，“证据”进入 evidenceModes。

66.1 [LAW-PROJECT-GOVERNANCE-001-V1] 权威顺序

要：必须按照现役说明书、基础法案、功能法案、按需施工图与规格、实现、当前证据的顺序裁决冲突。

不要：不得让任务提示词、代码、测试、generated artifact 或历史报告覆盖上级规范；不得在未获授权时修改 active law。

证据：static.text、static.contract。

违法后果：使用第 16 节 P0 固定文本。

66.2 [LAW-PROJECT-GOVERNANCE-002-V1] 唯一现役真相与 Owner

要：必须使每项生产事实、能力、合同、入口、registry、质量入口和路径具有一个现役真相源及一个 primary Owner。

不要：不得建立长期平行主链、第二 registry、第二 Owner map、第二 compiler 或第二质量 verdict。

证据：static.filesystem、static.dependency。

违法后果：使用第 16 节 P0 固定文本。

66.3 [LAW-PROJECT-GOVERNANCE-003-V1] 任务范围

要：必须使 READ_ONLY 直接读取且零落盘，并使 CHANGE 在首次写入前只记录 baseSha、allowedWritePaths 和 allowedDeletePaths，并在结束时核对实际差异。

不要：不得修改允许范围之外的路径或先写后补；不得覆盖用户原有改动或为只读任务建立 runtime 事务。

证据：static.filesystem、static.contract。

违法后果：使用第 16 节 P0 固定文本。

66.4 [LAW-PROJECT-GOVERNANCE-004-V1] 交付等级真实

要：必须对 CHANGE、READY、RELEASE 分级裁决，并明确每一级已完成和未完成内容。

不要：不得把阶段、MVP、preview、静态检查或部分证据冒充生产发布完成。

证据：static.text、static.contract。

违法后果：使用第 16 节 P0 固定文本。

66.5 [LAW-PROJECT-GOVERNANCE-005-V1] 有期限迁移

要：必须使迁移桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic 或 adapter 具有 Owner、监控、期限、删除日期、回滚条件和删除验收。

不要：不得形成两个长期真相源；不得到期不删；不得把迁移状态冒充最终终态。

证据：static.contract、static.config。

违法后果：使用第 16 节 P0 固定文本。

66.6 [LAW-PROJECT-GOVERNANCE-006-V1] 证据分层与禁止伪 PASS

要：必须使 Static、Exam、Dynamic 只证明各自实际观察对象，并使 READY、RELEASE 的逐 lawId 报告绑定当前等级和候选。

不要：不得用错误证据类型、历史结果、fixture、mock、warning、coverage 或自然语言声明放行。

证据：static.contract。

违法后果：使用第 16 节 P0 固定文本。

66.7 [LAW-PROJECT-GOVERNANCE-007-V1] 源头冻结与历史隔离

要：必须以 freeze 保护说明书，以当前 hash 绑定法案和 registry，并使 AGENTS 符合本说明书、历史只用于追溯。

不要：不得由普通任务自动改写权威；不得让历史文件、旧报告或 retired law 取得当前主权。

证据：static.config、static.json。

违法后果：使用第 16 节 P0 固定文本。

66.8 [LAW-PROJECT-GOVERNANCE-008-V1] 发布候选绑定

要：必须使 RELEASE 绑定 candidateId、Git commit、法案与 registry hash、逐法条报告、适用 Dynamic Proof 和跨仓发布结果。

不要：不得以 CHANGE、READY、旧候选证据或缺失外部证明的状态宣称 SYSTEM_PROVEN。

证据：static.contract。

违法后果：使用第 16 节 P0 固定文本。

67. 禁止递归元法案工厂

治理系统不得固定要求：

TXT

```
缺少独立长期风险依据的 law-system/audit-system 元法案；
每条法案一份 spec；
每条法案一个 blueprint anchor；
每条法案一个 predicate；
每条法案一个 Exam；
每个组件一个 mutant；
多份内容重叠的 generated projection；
多层 decision、receipt 和 report verdict。
```

新增治理 law 只有在第 66 节不能覆盖一个长期独立风险，且该风险可以写成可裁决规则时才合法。

业务 law 按 capabilityDomain 写入功能法案；其 blueprint、spec、Exam、Dynamic Proof 和专项测试均按实际拓扑、合同及 evidenceModes 建立。

68. 唯一命令面

跨实现稳定的公共治理命令身份只允许固定为：

TXT

```
governance:entry
governance:scope:check
audit:build
audit:projection:check
quality:check
```

它们分别承担入口检查、范围检查、registry 构建、registry 复验和唯一质量裁决。普通治理测试由 quality:check 调用，不冻结独立命令名。编程语言、内部脚本文件名、模块拆分、依赖和测试框架均由仓库决定，不属于本说明书冻结内容。

Exam 或 Dynamic Proof 存在时可以拥有独立执行命令，但只能把结果交给 canonical quality 汇总，不得形成第二质量 verdict。

第十部分 | 既有仓库迁移与新增能力

69. existing migration

既有仓库允许分步迁移。存在迁移桥时，必须在第 24 节 specKind=migration 的 spec 的 contract 中记录：

JSON

```
{
  "currentAuthority": "<identity>",
  "targetAuthority": "<identity>",
  "compatibilityMechanisms": [],
  "startDate": "YYYY-MM-DD",
  "monitoring": [],
  "rollbackCondition": "...",
  "maximumDurationDays": 30,
  "removalDate": "YYYY-MM-DD",
  "removalAcceptance": []
}
```

没有迁移桥的普通分步建设不创建 migration spec，不建立第二 runtime 任务记录。

maximumDurationDays 必须是 migration contract 按实际迁移声明的正整数；上例的 30 只是示例，不是全系统固定上限。removalDate 不得晚于 startDate 加 maximumDurationDays。

规则固定：

1. 中间步骤只能声明实际完成的 CHANGE 或 READY；最终步骤通过全部发布条件后才可声明 RELEASE。
2. 旧、新治理并存时必须指定当前裁决主权，不得双主权。
3. 临时 bridge、adapter 或 dual-read/write 依第 43.2 节登记和删除。
4. 完成最终 RELEASE 前，旧 normative source、旧 registry、旧 quality verdict 和过期 bridge 必须清零。
5. 治理迁移不得顺带修改未授权业务功能或工具链。

70. 新增 capabilityDomain

新增 domain 需要人类明确授权，并在首次生产写入前确定：

TXT

```
domain 名称；
primary Owner；
active lawIds；
ownedPaths；
与相邻 domain 的合同边界；
本阶段交付等级。
```

施工图、spec 依实际拓扑和合同建立，Exam、Dynamic Proof 只依 active law 的 evidenceModes 建立。不得因为尚未进入 RELEASE 而伪造空文件，也不得把未完成项写成已完成。

71. 新增 lawId

新增 lawId 必须同时完成：

TXT

```
唯一 law-record；
合法 lawId、domain 和当前版本关系；
Manifest.lawIds；
普通治理测试中的适用验证；
唯一 registry 重建；
进入 READY 或 RELEASE 时的逐 lawId 质量报告。
```

只有法条实际涉及机器合同才增加 spec；只有其 evidenceModes 声明对应类型时才增加 Exam 或 Dynamic Proof。不得机械创建一法一文件、一法一 predicate 或一法一 mutant。

72. lawId 修订和退役

72.1 修订

summary、forbidden、appliesTo/excludes、Owner、evidenceModes、tier 或依赖义务发生规范变化时，创建下一 major version并记录 supersedesLawId。

法条 major 与 spec version 独立：只改变业务合同而未改变法条义务时，只递增 spec version，不触发本说明书或无关 lawId 修订。破坏性合同变化可以在声明的兼容窗口内保留旧、新 spec version；必须满足第 43.2 节，并在期限内删除旧版本。兼容窗口不改变唯一规范主权：新版本生效范围和旧版本退休范围必须明确。

72.2 退役

退役必须删除 active block 和 Manifest 当前引用，移除独占入口，并通过唯一 builder 重建 registry 以移除当前记录；不得手改 generated registry。历史只保留在 Git 或明确 HISTORY_ONLY 路径。

被其它 active law、合同或消费者继续引用时不得宣称退役完成。

第十一部分 | 必须阻断的漂移

73. 架构漂移

必须阻断：

TXT

未经授权新增第二长期入口、第二 registry、第二 Owner map、第二 compiler 或第二质量 verdict；
把业务主权下沉到 shared/common/utils；
让 generated registry、Manifest、runner 或报告反向取得法案主权；
用临时迁移机制长期承载正式主链。

74. 范围漂移

必须阻断：

TXT

修改 change record 之外的路径；
为格式统一修改无关法案或 domain；
为未来支持增加未授权接口；
吞异常、缩小证据分母或覆盖用户原有改动；
发现旧代码后未经授权全面清理。

75. 证据漂移

必须阻断：

TXT

文件存在即 PASS；
token、数量或 Owner 存在即 PASS；
fixture 成功即生产成功；
历史报告即当前证据；
Static 即真实外部运行；
截图即完整动态证明；
CHANGE 或 READY 即 RELEASE。

76. 迁移漂移

迁移桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic、adapter 和兼容窗口本身不是违法。

必须阻断：

TXT

无 Owner；
无监控；
无回滚条件；
无最长期限或删除日期；
旧、新主权不明确；
到期未删除；
把迁移状态冒充最终 RELEASE；
以兼容为名永久保留两个真相源。

77. 文档漂移

必须阻断：

TXT

Codex 把自己的理解写成新法案；
在法条中加入背景故事、方案比较和开发日志；
在施工图或 spec 中复制法案；
在 AGENTS 或 README 维护第二法案清单；
在历史报告声明当前 READY、RELEASE 或 SYSTEM_PROVEN。

第十二部分 | 测试与独立自检

78. 普通测试与专项 mutation

第 51 节普通治理测试是默认质量手段。

只有 parser、registry builder、scope checker、quality aggregator 或禁止伪 PASS 的关键分支发生变化时，才要求对应专项 mutant。

mutation 规则：

TXT

```
一个 mutant 只改变一个关键条件；
必须触发预期失败；
在隔离目录运行；
不得改变原工作树；
不得为每条 law、每个 predicate、每个 component 机械配套 mutant；
不得以 mutant 数量或覆盖率作为 PASS。
```

RELEASE 执行当前候选中已建立且与变更的执法组件相交的 mutant；CHANGE 与 READY 只执行本次改变所影响的 mutant。

79. 独立自检

说明书定稿、治理主链变化或大规模迁移后，独立审查至少检查：

TXT

```
权威是否唯一；
任务范围是否越界；
registry 是否忠实投影 active laws；
逐法条报告是否使用正确证据；
CHANGE、READY、RELEASE 是否被如实区分；
迁移机制是否具有期限并可删除；
是否存在固定 PASS、遗漏分母或第二 verdict。
```

审查输出 finding，不以总体评价或覆盖率代替具体证据。

第十三部分 | 最终验收与冻结完成定义

80. 法案正文验收

TXT

active 法案容器精确两份；
基础法案完整包含第 66 节项目根规则与经人类明确授权加入的长期稳定治理 law；
功能法案包含实际业务与随能力持续迭代的治理 law；
全部 lawId、domain、supersedes 和当前版本关系成立；
forbidden 非空；
active law source 精确为固定两份；
不得存在只为递归证明治理实现自身而设立的元法案。

81. registry、施工图与规格验收

TXT

唯一 generated artifact 是 law-registry.generated.json；
registry 忠实投影全部 active laws、Owner 和实际 contract refs；
registry builder deterministic，projection check 只读；
施工图和 spec 只在实际复杂度要求时存在；
不存在一法一 spec、一法一 anchor 或空模板目录；
跨仓合同具有明确 provider、version 和 consumer pin。

82. Owner 与 scope 验收

TXT

每个生产路径和合同一个 primary Owner；
READ_ONLY 零 runtime 写入；
CHANGE record 只含 baseSha、allowedWritePaths、allowedDeletePaths；
实际差异未超出允许路径；
未获授权的 dirty target 未被触碰；
新增 surface 具有现役依据。

83. 交付与迁移验收

TXT

CHANGE、READY、RELEASE 分级明确；
阶段、MVP 和 preview 未冒充生产完成；
迁移桥具有 Owner、监控、回滚、期限、删除日期和删除验收；
跨仓通过版本化合同独立部署；
只有整体 RELEASE 要求必要参与仓发布结果；
外部 Dynamic Proof 不可用时只阻断 RELEASE，不阻断合规 CHANGE。

84. 质量验收

TXT

一套普通治理测试；
一份逐 lawId 质量报告；
READY 的推迟证据显式列出，RELEASE 的 deferredEvidenceModes 为空；
Static、Exam、Dynamic 不互相冒充；
Exam、Dynamic Proof 只按 active law 的 `evidenceModes` 建立，mutant 只按第 78 节的关键执法分支建立；
不存在平行 decision、receipt、report 或多层 verdict；
任一 finding、错误证据类型或 candidate drift 必须 FAIL；
CI 调用同一 canonical quality 入口；
RELEASE 才生成候选证明包。

85. 状态与完成定义

TXT

SPEC_FROZEN：说明书和启用的派生物由当前有效 freeze manifest 保护；
CHANGE_PASS：本次允许差异通过适用检查；
READY：候选通过声明的合并或部署前验收；
SYSTEM_PROVEN：当前 RELEASE candidate 的全部发布适用法条和证据通过。

最终运行公式：

```TXT
READ_ONLY：读取 -> 回答；
CHANGE：entry -> change record -> 修改 -> scope -> 受影响检查与普通测试；
READY：CHANGE -> 完整静态验收 -> 逐法条报告；
RELEASE：READY -> candidate binding -> 发布适用 Dynamic Proof -> 逐法条报告 -> proof package -> SYSTEM_PROVEN。
```

本说明书的最终原则：

> 治理只保留能够控制真实风险的规则。只读零事务，变更轻记录，发布强证明；允许小步建设和有期限迁移，但禁止越界、双重长期主权与伪 PASS。

附录 A | AGENTS.md 固定正文

bootstrap 生成的 AGENTS 规范性正文精确为：

MD

```
# Codex Governance Entry

本仓库使用冻结的法案系统。

1. READ_ONLY 直接读取并回答，不落盘、不加锁、不建立 runtime 任务事务。
2. CHANGE 首次写入前运行 `governance:entry`，只记录 baseSha、allowedWritePaths 和 allowedDeletePaths。
3. 修改不得超出 change record；扩大范围必须先更新同一 record。
4. READY 不等于 RELEASE；不得以静态检查、普通测试或部分证据冒充生产发布。
5. RELEASE 才绑定完整 candidateId，并执行发布适用的 Dynamic Proof。
6. 只允许一份 active law corpus、一份 law registry；每个生产路径和合同只允许一个 primary Owner；只允许一个 quality verdict。
7. 迁移桥、用于迁移的 feature flag、dual-read/write 或 shadow traffic 必须有 Owner、监控、期限、删除日期和删除验收。
8. active law 只在明确人类授权下修改。
9. 任一 finding、候选漂移、错误证据类型或伪 PASS 必须失败关闭。
```

AGENTS.md 中上述正文由本说明书授权，但不属于 active law source，也不进入 SPEC_FROZEN 字节绑定；其符合性由治理入口检查。

附录 B | 最小机器合同

B.1 change record

JSON

```
{
  "baseSha": "<git object id>",
  "allowedWritePaths": [],
  "allowedDeletePaths": []
}
```

只用于 CHANGE、READY、RELEASE 的路径控制；READ_ONLY 不创建。

B.2 Manifest

JSON

```
{
  "schemaVersion": 3,
  "capabilityDomain": "<lower-kebab-case>",
  "status": "active",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

Manifest 只登记 Owner 与路径，不登记逐法条 spec、anchor、predicate、Exam、mutant 或 report。

B.3 按需 blueprint machine block

JSON

```
{
  "schemaVersion": 1,
  "blueprintId": "<stable-id>",
  "status": "active",
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "contractRefs": [],
  "dependencyEdges": [
    {
      "from": {"repo": "local", "domain": "<capabilityDomain>"},
      "to": {"repo": "local", "domain": "<capabilityDomain>"},
      "contractRef": null
    }
  ],
  "prohibitedDependencyEdges": [
    {
      "from": {"repo": "local", "domain": "<capabilityDomain>"},
      "to": {"repo": "local", "domain": "<capabilityDomain>"},
      "lawId": "LAW-..."
    }
  ]
}
```

endpoint、item 字段、引用、排序、去重与冲突规则精确使用第 22 节。

B.4 按需 spec

JSON

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<stable-id>",
  "status": "active",
  "version": 1,
  "owner": "<capabilityDomain>",
  "lawIds": [],
  "specKind": "typed-contract",
  "contract": {}
}
```

B.5 law registry

JSON

```
{
  "schemaVersion": 3,
  "sourceSpec": {"path": "...", "sha256": "..."},
  "lawCorpus": {
    "baseLawPath": "...",
    "baseLawSha256": "...",
    "functionalLawPath": "...",
    "functionalLawSha256": "..."
  },
  "laws": [],
  "domains": []
}
```

laws[] 精确使用第 28 节 law record，domains[] 精确使用第 27 节 domain record。这是唯一 generated projection。

B.6 逐法条质量报告

JSON

```
{
  "schemaVersion": 1,
  "level": "RELEASE",
  "baseSha": "<git object id>",
  "candidateId": "<64 lowercase hex>",
  "registrySha256": "<64 lowercase hex>",
  "laws": [
    {
      "lawId": "LAW-...",
      "applicableEvidenceModes": [],
      "deferredEvidenceModes": [],
      "outcome": "PASS",
      "evidenceRefs": [],
      "findings": []
    }
  ],
  "outcome": "PASS"
}
```

这是唯一质量 verdict artifact。

B.7 跨仓 provider contract payload

本节不是独立 artifact；它是第 B.4 节 spec 在 specKind=cross-repo-contract 时的 contract 内容：

JSON

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo",
  "providerDomain": "provider-domain",
  "authorityArtifact": {
    "path": "repo/relative/path",
    "selector": "stable-selector",
    "sha256": "<64 lowercase hex>"
  },
  "consumerRepos": []
}
```

B.8 跨仓 consumer pin payload

本节不是独立 artifact；它是第 B.4 节 spec 在 specKind=contract-pin 时的 contract 内容：

JSON

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo",
  "providerDomain": "provider-domain",
  "providerSpecRef": "docs/specs/<provider-domain>/<spec-id>.spec.json",
  "providerCommitSha": "<git object id>",
  "contractSha256": "<64 lowercase hex>"
}
```

consumerRepos[] item 必须是 canonical owner/repo 字符串，按 UTF-8 bytes 排序且无重复；B.7、B.8 对象均拒绝未知字段。provider 与 consumer 可以独立部署；合同内容变化递增 version，破坏性改变同时声明兼容窗口和迁移期限。

contractSha256 绑定 provider spec 的 contract canonical JSON bytes；consumer pin 的 contractId、version、providerRepo、providerDomain 和该 hash 必须与 provider spec 完全一致。providerCommitSha 绑定包含该 provider spec 的提交。

B.9 RELEASE proof package

JSON

```
{
  "schemaVersion": 1,
  "candidateId": "<64 lowercase hex>",
  "gitCommit": "<git object id>",
  "freezeSha256": "<64 lowercase hex>",
  "baseLawSha256": "<64 lowercase hex>",
  "functionalLawSha256": "<64 lowercase hex>",
  "registrySha256": "<64 lowercase hex>",
  "lawQualityReportSha256": "<64 lowercase hex>",
  "dynamicProofHashes": [],
  "crossRepoParticipants": [
    {
      "contractId": "provider-domain.contract",
      "contractVersion": 1,
      "repo": "owner/repo",
      "releaseSha256": "<64 lowercase hex>"
    }
  ]
}
```

proof package 只在 RELEASE PASS 后生成；不得用于 CHANGE 或 READY。

proof package 顶层及 participant item 均拒绝未知字段；crossRepoParticipants[] 按 contractId、contractVersion、repo 排序且无重复，并按第 30 节生成 canonical JSON。参与仓依赖必须无环；每个 package 只引用已经完成 RELEASE 的直接依赖仓 package，不得形成互相 hash 的递归绑定。

hash 身份固定为：

TXT

freezeSha256：当前有效 docs/governance/GOVERNANCE-FREEZE.json 原始字节的 SHA-256；
baseLawSha256、functionalLawSha256：对应 active law 文件原始字节的 SHA-256；
registrySha256：唯一 registry canonical JSON bytes 的 SHA-256；
lawQualityReportSha256：本次 RELEASE 逐法条质量报告 canonical JSON bytes 的 SHA-256；
dynamicProofHashes[]：逐法条质量报告实际引用的 Dynamic Proof artifact 的 SHA-256，去重后按小写 hash 升序排列；
crossRepoParticipants[].releaseSha256：对应参与仓 RELEASE proof package canonical JSON 原始字节的 SHA-256。

不得为解释这些 hash 另建 receipt。